Guia de Apresentação

Simulador de Zeros de Funções — Cálculo Numérico, UNIVASF 2026.2

Este guia diz o que cada integrante precisa saber para explicar o trabalho e responder perguntas. **Todos precisam estar presentes** (13 ou 15/10/2026) e o professor pode perguntar qualquer parte a qualquer pessoa. Por isso, a seção 1 vale para os três; as seções 2, 3 e 4 são o aprofundamento de cada parte, mas todos devem ler as três.

1. O que os três precisam saber

1.1 O problema

Achar o zero de f(x), isto é, o x em que f(x) = 0. Nos problemas: em que instante o componente chega a 50 graus C (Problema 1) e em que instante o capacitor chega a 3,8 V (Problema 2). O programa roda quatro métodos iterativos no mesmo problema e compara o desempenho.

1.2 Os quatro métodos em uma frase

Método	Ideia	Precisa de	Convergência
Bissecção	Corta o intervalo ao meio e fica com a metade onde há troca de sinal.	[a, b] com f(a)*f(b) < 0	Linear, lenta, sempre converge
Falsa Posição	Traça a reta entre (a, f(a)) e (b, f(b)); o x é onde ela cruza o eixo.	[a, b] com f(a)*f(b) < 0	Linear (costuma ser mais rápida que a bissecção, mas pode travar uma extremidade)
Newton-Raphson	Usa a reta tangente no ponto atual: x_novo = x - f(x)/f'(x).	x0 e f'(x)	Quadrática, muito rápida, pode falhar
Secante	Como o Newton, mas troca f' pela inclinação entre os dois últimos pontos.	x0 e x1	Superlinear (cerca de 1,618), não precisa de f'

1.3 Combinados do projeto (podem ser perguntados)

- **Critério de parada igual nos 4:** |f(x)| < eps, com eps = 1e-6. O enunciado confere exatamente esse resíduo.
- **Mesmo eps e max_iter (100)** em todos os métodos, para a comparação ser justa.
- **Saiu do intervalo não é falha:** o método continua e a tabela mostra um aviso (o enunciado pede para registrar a extrapolação).
- **Todo método devolve um Resultado** (resultado.py): método, raiz, iterações, tempo, resíduo, erro, aviso e histórico. Nenhum método usa print ou input; quem mostra é o relatorio.py.
- **Tempo** medido com time.perf_counter(), em milissegundos.

1.4 Resultados que vocês devem saber de cabeça

Caso	Bissecção	Falsa Posição	Newton	Secante
cos(x) - x, [0,1]	20 it.	5 it.	3 it.	4 it.
Problema 1, [0,20]	26 it.	23 it.	5 it.	10 it. (saiu do intervalo)
Problema 2, [0,3]	21 it.	25 it.	5 it.	9 it.

Em todos os casos, Newton foi o mais rápido em iterações. Todos atingiram |f(raiz)| < 1e-6. Raízes de referência: 0,7390851332 (exemplo), 7,3240819245 (Problema 1, igual a ln(3)/0,15) e 1,4271163556 (Problema 2, igual a -ln(0,24)).

1.5 Como demonstrar

- Rodar `python main.py`, escolher o exemplo e depois um problema.
- Mostrar a tabela no terminal, abrir a janela de gráficos e, se der tempo, a animação de um método.
- Mostrar uma falha de propósito: digitar uma função sem mudança de sinal no intervalo e ver o erro tratado.
- **Antes de apresentar:** `pip install -r requirements.txt` no computador usado e um teste completo. Deixar os vídeos de docs/animacoes/ como plano B.

2. Parte 1 — Métodos de intervalo (Caio Benevides)

Arquivo: **metodos_intervalo.py** (funções *bisseccao* e *falsa_posicao*).

2.1 O que explicar

- **Teorema de Bolzano:** se f é contínua em [a, b] e f(a)*f(b) < 0, existe ao menos uma raiz dentro. É por isso que os dois métodos exigem mudança de sinal.
- **Bissecção:** x = (a+b)/2. Se f(a)*f(x) < 0 a raiz está em [a, x] (b = x); senão está em [x, b] (a = x). O intervalo cai pela metade a cada passo.
- **Falsa Posição:** x = (a*f(b) - b*f(a)) / (f(b) - f(a)), onde a reta entre os extremos cruza o eixo. Atualiza o intervalo do mesmo jeito.
- **Por que sempre convergem:** a raiz fica presa no intervalo, que só encolhe. **Por que são lentos:** convergência linear e não usam a derivada nem a forma da curva (a bissecção ignora até os valores de f, só usa o sinal).

2.2 Falhas tratadas no código

- **Sem mudança de sinal:** f(a)*f(b) >= 0 devolve erro antes de iterar.
- **Divisão por zero na Falsa Posição:** se f(b) - f(a) = 0, devolve erro.
- **Não convergiu em max_iter:** devolve a última aproximação com mensagem de erro.

2.3 Perguntas prováveis e respostas

Pergunta	Resposta
Quantas iterações a bissecção precisa?	O erro cai pela metade a cada passo: n é cerca de log2((b-a)/eps). Para [0,1] e 1e-6 dá uns 20, o que bate com o resultado (20).
Por que a Falsa Posição levou 25 no Problema 2 e só 5 no exemplo?	Em funções com curvatura forte, uma extremidade tende a ficar parada e a reta secante melhora devagar. No exemplo cos(x)-x a curva é quase uma reta, e ela acerta rápido.
E se houver duas raízes no intervalo?	Se f(a)*f(b) < 0 há um número ímpar de raízes e o método acha uma delas. Se for par, o teste de sinal nem detecta (f(a)*f(b) > 0).
Por que testar f(a)*f(b) >= 0 e não só < 0?	Igual a zero significa que uma ponta já é raiz; o programa trata como entrada sem troca de sinal, por simplicidade. O produto pode dar overflow/underflow em casos extremos; comparar os sinais seria mais robusto.
Qual a diferença entre erro e resíduo?	O critério usa o resíduo |f(x)|, não o erro |x - raiz|. Um resíduo pequeno não garante erro pequeno se a curva for muito plana perto da raiz.

3. Parte 2 — Métodos abertos (Tãua Oliveira)

Arquivo: **metodos_abertos.py** (funções *newton_raphson* e *secante*).

3.1 O que explicar

- **Newton-Raphson:** x_novo = x - f(x)/f'(x). Desenha a tangente no ponto atual e pega onde ela cruza o eixo x. Convergência quadrática: perto da raiz, o número de casas corretas dobra a cada passo (por isso 3 a 5 iterações).
- **Secante:** x_novo = x1 - f(x1)*(x1 - x0)/(f(x1) - f(x0)); depois x0 = x1 e x1 = x_novo. Aproxima a derivada pela inclinação entre dois pontos. Não precisa de f', mas é um pouco mais lenta que o Newton.
- **Por que são rápidos, mas podem falhar:** não prendem a raiz num intervalo. Se o chute for ruim, podem divergir, oscilar, cair em outra raiz ou sair do domínio da função.
- **Derivada:** no programa, f'(x) é calculada automaticamente com sympy (entrada.py). Nos problemas, bate com a do enunciado (-13,5e^(-0,15t) e 5e^(-t)).

3.2 Falhas tratadas no código

- **Derivada nula (Newton):** |f'(x)| < 1e-14 devolve erro (tangente horizontal nunca cruza o eixo).
- **Divisão por zero (Secante):** |f(x1) - f(x0)| < 1e-14 devolve erro.
- **Saída do intervalo:** registra em que iteração e com que valor saiu, mas continua. A raiz final fora do intervalo também gera aviso.
- A tolerância 1e-14 é só para decidir se pode dividir; é diferente do eps de convergência.

3.3 Ponto que o professor deve notar

Problema 1, Secante: com x0 = 0 e x1 = 20 ela foi para t = -3,4757 na iteração 2 (fora do intervalo e fora de t >= 0) e depois voltou, convergindo em 10 iterações. Isso é a *extrapolação do intervalo* que o enunciado pede para explicar. Os dois pontos iniciais ficam em lados opostos da raiz, mas a reta secante entre eles, numa curva convexa como a exponencial, cruza o eixo fora do intervalo. Newton, partindo de t0 = 0, não saiu.

3.4 Perguntas prováveis e respostas

Pergunta	Resposta
Por que o Newton é tão rápido?	Convergência quadrática: o erro do próximo passo é proporcional ao quadrado do erro atual.
Quando o Newton falha?	Derivada zero ou quase zero, chute longe da raiz, função com ponto de inflexão ou oscilação (o passo cai em ciclo), ou raiz múltipla (perde a velocidade).
Por que usar a Secante se o Newton é mais rápido?	Quando f' é difícil ou cara de calcular. Cada passo da Secante reaproveita o valor anterior e só avalia f uma vez.
Por que x0 = 0 no Newton do Problema 2?	O enunciado manda usar uma das extremidades. Em 0, f' = 5, bem diferente de zero, e a função é côncava, então converge bem.
Se o Newton começasse em um ponto com f' = 0?	Falha controlada: devolve 'Derivada nula', sem travar o programa. Há um teste disso em testes.py (x0 = 0 em uma função com f'(0) = 0).

4. Parte 3 — Interface e saída (Giovanni Éber)

Arquivos: **main.py, entrada.py, casos.py, comparacao.py, relatorio.py, animacao.py**.

4.1 Fluxo do programa

main.py (menu) → casos.py ou entrada.py (dados) → comparacao.py (roda os 4 métodos com o mesmo eps e max_iter) → relatorio.py (tabela e janela de gráficos) → animacao.py (animação opcional). O diagrama está em docs/fluxo_dados.png e pode ir no slide.

4.2 O que explicar

- **Menu:** escolhe um caso pronto (exemplo, Problema 1 ou 2) ou digita outra função; depois lê eps e max_iter (Enter usa o padrão) e volta ao menu: atende 'novo cálculo ou sair'.
- **Entrada com sympy:** o texto digitado (ex.: cos(x) - x) vira expressão, a variável é detectada (x, t...) e a derivada é calculada automaticamente com sympy.diff. Aceita ^ como potência, 'sen' e 'ln'.
- **Validação:** ler_float aceita vírgula decimal e notação científica e rejeita inf/nan; ler_int exige inteiro maior que 0; função com 0 ou mais de 1 variável gera mensagem amigável em vez de quebrar.
- **Segurança de execução:** comparacao.py captura erros matemáticos (log de negativo, overflow, divisão por zero) e transforma em falha do método, sem derrubar o programa.
- **Tabela e gráficos:** tabela com método, raiz, iterações, tempo e |f(raiz)|; asterisco e nota de rodapé quando há aviso; janela matplotlib com a curva, as raízes e a convergência (bonificação do enunciado).
- **Animação:** seletor com um quadrado por método; mostra as iterações passo a passo. Vídeos prontos em docs/animacoes/.

4.3 Relevância do modelo (pedido no Problema 2)

A tensão V(t) = 5(1 - e^(-t)) descreve a carga de um capacitor num circuito RC. Achar o instante em que V chega a 3,8 V diz *quando um limiar de tensão é atingido*: é a base de temporizadores, filtros de sinal, leitura de sensores e atrasos em sistemas embarcados. Nesse tipo de projeto, calcular o tempo certo define o desempenho e a segurança do circuito. O Problema 1 é análogo: tempo para o componente resfriar até uma temperatura segura.

4.4 Perguntas prováveis e respostas

Pergunta	Resposta
Como o programa aceita outras funções?	Pela entrada de texto com sympy; o usuário digita a função e os dados iniciais, e a derivada sai automaticamente.
Como vocês garantiram uma comparação justa?	Mesmos eps (1e-6), max_iter (100) e critério de parada nos 4 métodos.
Por que o tempo variou entre execuções?	São centésimos de milissegundo; o sistema operacional gera ruído. A análise prioriza iterações e resíduo, como o enunciado orienta.
Como testaram?	testes.py: 36 verificações, com os três casos, raiz comparada com a de referência, resíduo menor que eps, contagem de iterações e os 4 tratamentos de falha.
O que acontece com entrada inválida?	O programa avisa e pergunta de novo; nunca encerra por erro de digitação.

5. Roteiro sugerido (cerca de 10 minutos)

Tempo	Quem	Conteúdo
1 min	Giovanni	Objetivo, os quatro métodos e a estrutura do programa (diagrama de fluxo).
2 min	Caio	Bissecção e Falsa Posição; Bolzano; falhas tratadas.
2 min	Tãua	Newton e Secante; derivada; falhas e extrapolação do intervalo.
3 min	Giovanni	Demonstração ao vivo: exemplo, um problema, tabela, gráfico e uma falha.
2 min	Todos	Análise: Newton foi o mais rápido; intervalo é seguro, mas lento; qual usar quando.

5.1 Conclusão para fechar

- Métodos de intervalo: garantia de convergência, mas lentos (20 a 26 iterações).
- Métodos abertos: rápidos (3 a 10 iterações), mas dependem do chute inicial e, no Newton, da derivada.
- Boa prática: usar um método de intervalo para localizar a raiz e um aberto para refinar.
- Todos atingiram |f(raiz)| < 1e-6 nos três casos.

5.2 Checklist do dia

- Computador com Python, sympy e matplotlib instalados, e o código atualizado (git pull).
- Testar o programa inteiro uma vez, incluindo a janela de gráficos.
- Vídeos de docs/animacoes/ abertos em outra aba, como plano B.
- Cada um sabe responder pelas três partes, não só pela sua.
- Relatório em PDF: entrega até 16/10/2026.